

Table des matières

1	Diagramme des cas d'utilisation	3
1.1	Objectif du diagramme	4
1.2	Acteurs identifiés	4
1.3	Structuration fonctionnelle	4
1.4	Relations entre cas d'utilisation	5
1.5	Cohérence d'ensemble	5
2	Classes	7
2.1	Diagramme de classes	7
2.2	Détails des méthodes	7
3	Diagrammes de séquence	10
4	Tests d'intégrations	18
4.1	Gestion de la partie (10 tests)	18
4.2	Gestion des joueurs (8 tests)	20
4.3	Gestion économique (20 tests)	22
4.4	Gestion du plateau (23 tests)	28
4.5	Gestion des mini-jeux (12 tests)	33
4.6	Robustesse (9 tests)	36

Introduction

Ce document est à destination du client. Par la suite, nous nous allons proposer une modélisation UML de notre jeu. Dans un premier temps, nous énoncerons les différents cas d'utilisation. Ensuite, bien que la place d'un tel diagramme dans le cahier de spécifications soit discutable, nous voulons rendre la modélisation de notre jeu la plus claire possible pour le client.

1 Diagramme des cas d'utilisation

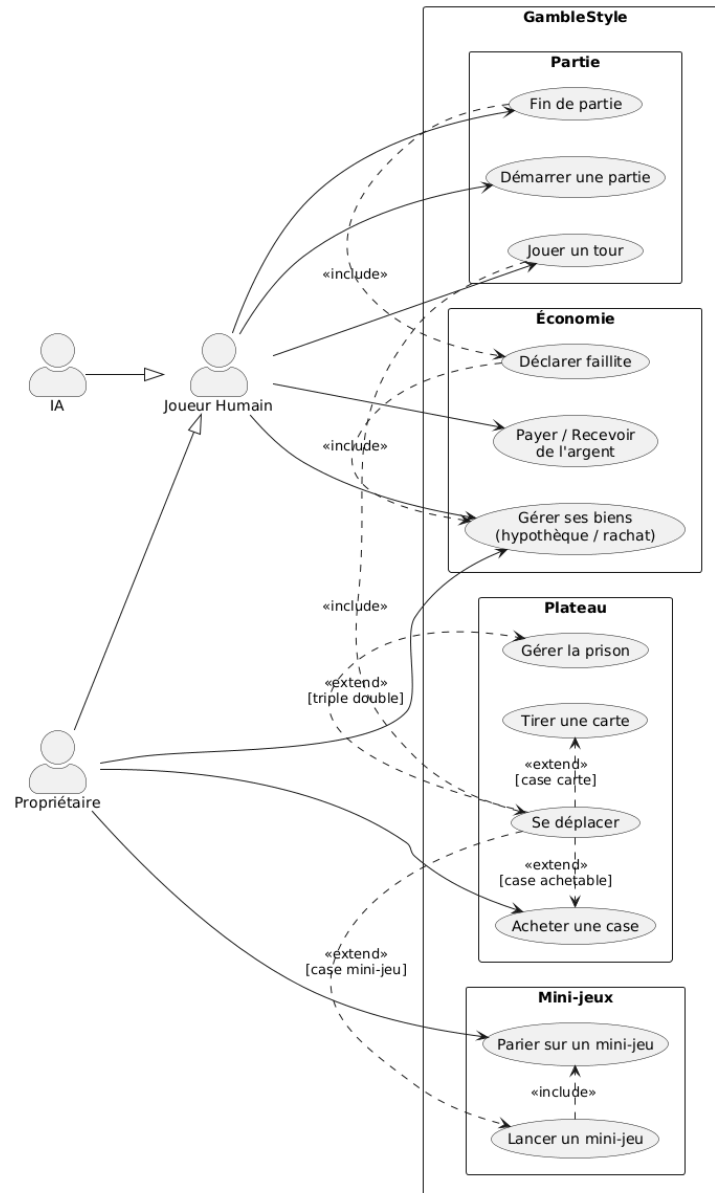


FIGURE 1 – Diagramme des cas d'utilisation

1.1 Objectif du diagramme

Le diagramme des cas d'utilisation a pour objectif de représenter, du point de vue externe, les grands services rendus par le système *GambleStyle* ainsi que les rôles qui interagissent avec lui. Ce choix est cohérent avec la démarche UML attendue dans le projet, où le diagramme de cas d'utilisation doit expliciter les fonctionnalités principales du système, les acteurs et leurs interactions avant la modélisation détaillée des classes et des scénarios dynamiques.

1.2 Acteurs identifiés

L'acteur principal est le **Joueur Humain**, car l'essentiel des fonctionnalités décrites dans le cahier des charges lui sont directement accessibles. C'est lui qui démarre une partie, joue un tour, se déplace sur le plateau, achète éventuellement une case, gère son argent, ses biens et participe aux mini-jeux. Il constitue donc l'acteur générique représentant tout participant soumis aux règles générales du jeu.

L'**IA** est modélisée comme une spécialisation du Joueur Humain. Ce choix se justifie par le fait que le projet prévoit explicitement la possibilité de jouer avec des intelligences artificielles. Une IA réalise en effet les mêmes actions de jeu qu'un joueur classique ; la différence ne porte pas sur les fonctionnalités disponibles, mais sur la manière dont les décisions sont prises. La relation de généralisation permet donc d'exprimer que l'IA hérite des mêmes cas d'utilisation que le joueur humain.

Le **Propriétaire** est lui aussi représenté comme une spécialisation du Joueur Humain. Il ne s'agit pas d'un acteur entièrement distinct, mais d'un joueur qui acquiert, au cours de la partie, un statut particulier par la possession de cases. Ce statut lui donne des responsabilités et des interactions supplémentaires, notamment dans la gestion de ses biens, dans certains transferts économiques, et lors du déclenchement des mini-jeux lorsqu'un autre joueur tombe sur une case lui appartenant.

1.3 Structuration fonctionnelle

Le système a été structuré en quatre sous-ensembles fonctionnels : **Partie**, **Économie**, **Plateau** et **Mini-jeux**. Cette décomposition reprend les grands blocs fonctionnels définis dans le cahier des charges et permet de distinguer clairement les responsabilités du système.

- **Partie** : pilotage du déroulement global d'une session.
- **Économie** : gestion des flux d'argent, des biens et de la faillite.
- **Plateau** : traitements liés aux déplacements et aux types de cases.
- **Mini-jeux** : affrontements déclenchés selon les événements du plateau.

Le paquet **Partie** regroupe les cas d'utilisation qui pilotent le déroulement global d'une session. Le cas *Démarrer une partie* correspond à l'initialisation du jeu : choix du nombre de joueurs, mise en place des paramètres initiaux, détermination de l'ordre des tours et lancement de la partie. Le cas *Jouer un tour* traduit la boucle élémentaire du gameplay, puisqu'une partie se construit par succession de tours. Le cas *Fin de partie* représente la détection de la condition d'arrêt et la désignation du gagnant lorsque tous les autres joueurs sont en faillite, ou lors d'un arrêt manuel avec comparaison des patrimoines.

Le paquet **Économie** isole toutes les mécaniques monétaires et patrimoniales. Le cas *Payer / Recevoir de l'argent* est central, car il intervient dans de nombreuses situations : gains liés au passage sur la case départ, pertes ou gains dus aux cartes, résultat d'un mini-jeu, paiement d'une mise ou transfert

entre joueurs. Le cas *Gérer ses biens (hypothèque / rachat)* est distingué car il correspond à une action patrimoniale spécifique permettant à un joueur de retrouver de la liquidité. Enfin, le cas *Déclarer faillite* représente le cas limite de l'économie du jeu, lorsque le joueur ne peut plus honorer ses dettes et que son capital de solvabilité devient nul.

Le paquet **Plateau** regroupe les interactions directement liées à la position du joueur et à la nature de la case atteinte. Le cas *Se déplacer* est ici fondamental, car un tour conduit nécessairement à une évolution de la position du joueur sur le plateau. À partir de ce déplacement, plusieurs traitements spécialisés peuvent être déclenchés selon le type de case : *Acheter une case* lorsqu'une case achetable est atteinte, *Tirer une carte* lorsqu'il s'agit d'une case aléatoire, ou *Gérer la prison* lorsqu'une règle particulière liée à la prison s'applique.

Le paquet **Mini-jeux** correspond à l'une des spécificités majeures de *GambleStyle*. Le cas *Lancer un mini-jeu* formalise le déclenchement et l'exécution du mini-jeu, soit en duel lorsqu'un joueur tombe sur une case appartenant à un autre joueur, soit avec l'ensemble des joueurs lorsqu'une case événement est rencontrée. Le cas *Parier sur un mini-jeu* est distingué afin de rendre visible la couche économique supplémentaire qui entoure les mini-jeux, puisque les joueurs non directement impliqués peuvent miser sur l'issue d'un affrontement.

1.4 Relations entre cas d'utilisation

Les relations **include** traduisent des sous-traitements nécessaires à l'exécution d'un cas d'utilisation plus large. Ainsi, jouer un tour implique nécessairement certains traitements du plateau et, selon les situations, des opérations économiques. De même, le lancement d'un mini-jeu s'accompagne d'une préparation liée aux mises et aux paris, ce qui justifie le lien fort entre les cas d'utilisation du paquet *Mini-jeux* et ceux du paquet *Économie*.

Les relations **extend** modélisent des comportements conditionnels. Ainsi, *Acheter une case* étend *Se déplacer* uniquement dans le cas où la case atteinte est achetable. De la même manière, *Tirer une carte* n'intervient que lorsque le joueur arrive sur une case de type carte. Le déclenchement d'un mini-jeu peut aussi être compris comme une extension du déplacement, lorsqu'un joueur arrive sur une case mini-jeu appartenant à un autre joueur ou sur une case événement. Enfin, *Gérer la prison* intervient dans des situations particulières du tour, par exemple lors d'une arrivée sur la case prison ou en cas de triple double.

1.5 Cohérence d'ensemble

Le diagramme est globalement cohérent car il fait apparaître *Jouer un tour* comme noyau dynamique du jeu, autour duquel viennent s'articuler les dimensions de déplacement, d'économie et de compétition par mini-jeux. Il montre aussi qu'un même acteur peut changer de rôle au cours de la partie, notamment en devenant propriétaire. Cette modélisation prépare naturellement la suite de la conception orientée objet, car les cas d'utilisation identifiés ici correspondent aux scénarios détaillés ensuite dans les diagrammes de séquence et aux responsabilités portées par les futures classes du système.

En définitive, ce diagramme des cas d'utilisation constitue une synthèse fonctionnelle pertinente du projet *GambleStyle*. Il respecte la logique attendue d'une analyse UML en identifiant les acteurs, les services rendus par le système et les dépendances entre ces services. Il reflète également fidèlement la

structure métier du jeu, qui repose sur l'enchaînement des tours, la gestion du plateau, l'économie des propriétés et l'intégration des mini-jeux.

2 Classes

2.1 Diagramme de classes

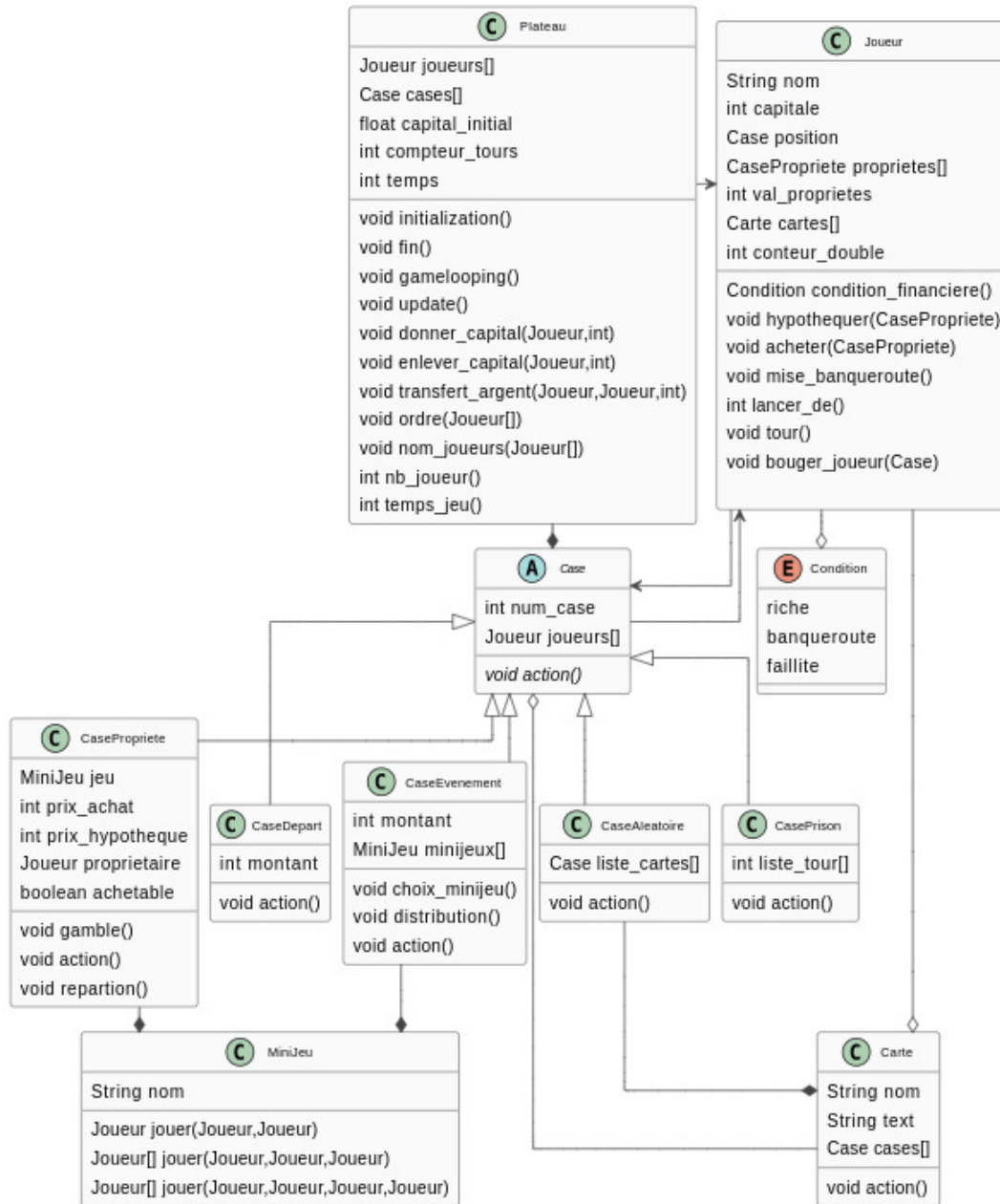


FIGURE 2 – Diagramme de classes

2.2 Détails des méthodes

Classe Joueur :

- *Condition condiation_financiere()* : obtenir la condition d'un joueur (la Condition est un type d'énumération, renvoie 'faillite', 'banqueroute', ou 'riche'). Voir cahier des charges sur comment obtenir la condition d'un joueur.
- *void hypotequer(CasePropriete)* : en appelant cette méthode, le joueur choisi de faire hypothéquer une de ses cases.
- *void acheter(CasePropriete)* : lorsque le joueur atterri sur une case CasePropriete, qui est achetable, et que le joueur souhaite l'acheter, alors cette méthode est à utiliser. Ajouter alors la CasePropriete à la suite de son attribut 'proprietes', retirer le capital du prix d'achat, ajouter le capital à 'val_propriete'
- *void mise_banqueroute()* : met le joueur en banqueroute. Il faudra alors lui proposer de mettre des bien à hypothéquer jusqu'à ce qu'il sorte de la banqueroute, et redevienne dans une condition "riche". Si cela n'est pas possible, alors le joueur passe en faillite. Il perd ses proprietes et son capital.
- *int lancer_de()* : renvoi un nombre aléatoirement entre 1 et 6
- *void tour()* : utiliser 'lancer_de()' deux fois pour simuler une lancée de deux dés. Si cela donne un double, incrémenté l'attribut 'compteur_de' d'un. faire avancer le joueur jusqu'à la case associée. S'il y a eu un double, recommencer. Jusqu'à maximum 3 doubles. Dans ce cas, envoyer joueur en prison. 0 la fin, remettre 'compteur_tour' à 0.
- *void bouger_joueur(Case)* : faire aller le joueur jusqu'à la case associée. Actionner la case.

Classe Plateau :

- *void ordre()* : faire lancer les dés à chacun des joueurs (du tableau d'attribut 'joueurs') pour déterminer l'ordre des joueurs dans un tour. Réorganise le tableau de joueurs afin que le premier joueur soit le premier de la liste, le deuxième en deuxième etc.
- *int nb_joueur()* : demander au terminal/ interface graphique combien de joueurs vont jouer
- *void nom_joueurs()* : demander au terminal/interface graphique le nom des joueurs qui vont jouer. Mettre les noms dans l'attribut 'joueurs'.
- *void temps_jeu()* : demander au terminal / interface graphique combien de minutes les joueurs veulent jouer au maximum.
- *void update()* : fait jouer les joueurs 1 tour dans l'ordre
- *void gamelooping()* : faire boucler 'update()' jusqu'à ce que le jeu soit terminé
- *void Initialisation()* :
 - demander le nombre de joueur, leur nom, leur faire tirer les dés pour connaître leur ordre de passage. Avec ces informations, on pourra compléter l'attribut 'joueurs = [joueur1, joueur2,...]'. Demander le temps de jeu.
 - Faire jouer les joueurs avec 'gamelooping()'
 - fin de la partie, annonce du gagnant
- *void fin()* : faire terminer la partie. Récupérer par rapport à leur capital + biens qui est gagnant numéro 1, puis numéro 2... C'est l'annonce des gagnants.
- *void donner_capital(Joueur, montant)* : ajouter le capital au joueur.
- *void enlever_capital(Joueur, montant)* : enlever le capital au joueur. Mise en banqueroute si besoin.
- *transfert_argent(Joueur1, Joueur2, montant)* : combiner 'donner_capital(Joueur2, montant)' et 'enlever_capital(Joueur1, montant)'.

Classe Carte :

- *void action()* : fait l'action de la carte. Voir cahier des charges pour les différentes cartes à coder. Pourra soit modifier le capital des joueurs, soit modifier la position d'un joueur sur le plateau,

soit une carte qui sera possédée par un joueur, que celui-ci pourra utiliser aux moments qu'il souhaitera.

Classe Case :

- *void action()* : est une méthode abstraite. Une méthode *void action()* sera codée par classe fille de Case.

Classe CaseAleatoire :

- *void action()* : parmi la liste avec les cartes, on prend la première. Puis on fera *Carte.action()* pour activer cette carte. On place cette première carte à la queue de la liste, afin de ne pas l'avoir de nouveau ensuite.

Classe CaseEvenement :

- *void action()* : lorsque *action()* est activé, un mini-jeu sera choisis au hasard à jouer au nombre total du jeu. Il faut y jouer, puis distribuer la mise en fonction du gagnant.
- *void distribution()* : distribue la mise après jeu en fonction des gagnants.
- *void choix_minijeu()* : choisit un mini-jeu au hasard (tous les joueurs doivent y jouer).

Classe CaseDepart :

- *void action()* : donner la somme montant (attribut de la classe *CaseDepart*) au joueur passant par la case.

Classe CasePropriete :

- *void gamble()* : demander au propriétaire de la case (s'il y en a un) combien il veut gamble sur cet minijeu avant d'y jouer.
- *void repartition()* : Récupération de la connaissance du joueur gagnant. Enlever la valeur du gamble au perdant pour l'ajouter au capital du gagnant.
- *void action()* : si la case en question possède déjà un propriétaire, lancement du gamble et du mini-jeu. Si la case est achetable, demander au joueur s'il veut l'acheter.

Classe CasePrison :

- *void action()* : Si le joueur possède une carte 'sortiePrison', lui demander s'il veut la jouer. Si oui, ne rien faire d'autre. Sinon : faire lancer les dés au joueur. S'il fait un double, ne rien faire. S'il ne fait pas un double, incrémenter de 1 dans sa position dans *liste_tour[]*. Quand *liste_tour[int joueur] = 3*, mettre *liste_tour[int joueur] = 0*. Cela signifie que le joueur pourra continuer de lancer les dés etc.

Classe Minijeu :

- *Joueur jouer(Joueur,Joueur)* : lancer le jeu à jouer avec les deux Joueurs. Renvoyer le joueur qui a gagné le minijeu
- *Joueur[] jouer(Joueur,Joueur,Joueur)* : lancer le jeu à jouer avec les trois Joueurs. Renvoyer la liste des 3 joueurs trié dans l'ordre. En 0 celui qui gagne, en 1 le deuxième. En 2 le perdant.
- *Joueur[] jouer(Joueur,Joueur,Joueur,Joueur)* : Idem que précédemment avec quatre joueurs.

Classe Carte :

- *void action()* : chaque carte aura une action avec ce que fera l'action. Ce sera soit des échanges de capital, soit un déplacement de case... On aura plusieurs instances de Carte. Voici deux exemples : une carte Anniversaire, où *action()* fera un transfert d'argent de tous les joueurs vers celui qui aura obtenu la carte. C'est une carte de type transfert d'argent. (tout comme les cartes Impot, gain...) Il y a aussi le type de carte de déplacement dans le plateau. Et enfin, il y a les cartes que l'on possède (carte prison, carte double mise).

3 Diagrammes de séquence

Les diagrammes de séquence présentés dans ce document visent à décrire de manière dynamique le fonctionnement du système Gamble Style, en mettant en évidence les interactions entre les différents objets au cours du temps. Contrairement au diagramme de classes, qui représente la structure statique du système, ces diagrammes permettent de modéliser concrètement les scénarios d'exécution associés aux cas d'utilisation identifiés lors de l'analyse fonctionnelle. Le diagramme *Déroulement d'un tour complet* représente le déroulement d'un tour de jeu, depuis l'activation de la boucle principale jusqu'au passage au joueur suivant.

Le diagramme *Achat d'une case propriété* décrit le cas où un joueur arrive sur une case achetable et décide d'acheter (ou non) la propriété.

Le diagramme *Déclenchement d'un mini-jeu* modélise l'interaction entre le propriétaire d'une case et le joueur visiteur, qui doivent participer à un mini-jeu après l'acquisition ou lors d'une visite.

Le diagramme *Case événement* illustre le comportement du système lorsqu'un joueur tombe sur une case événement, entraînant l'activation d'un mini-jeu impliquant plusieurs joueurs.

Enfin, le diagramme *Gestion de la banqueroute / faillite* décrit la manière dont le système gère les situations de dette, de remboursement et d'élimination d'un joueur en cas de faillite.

Ces diagrammes permettent de couvrir les principales mécaniques du jeu et de garantir la cohérence des interactions entre les différents composants du système.

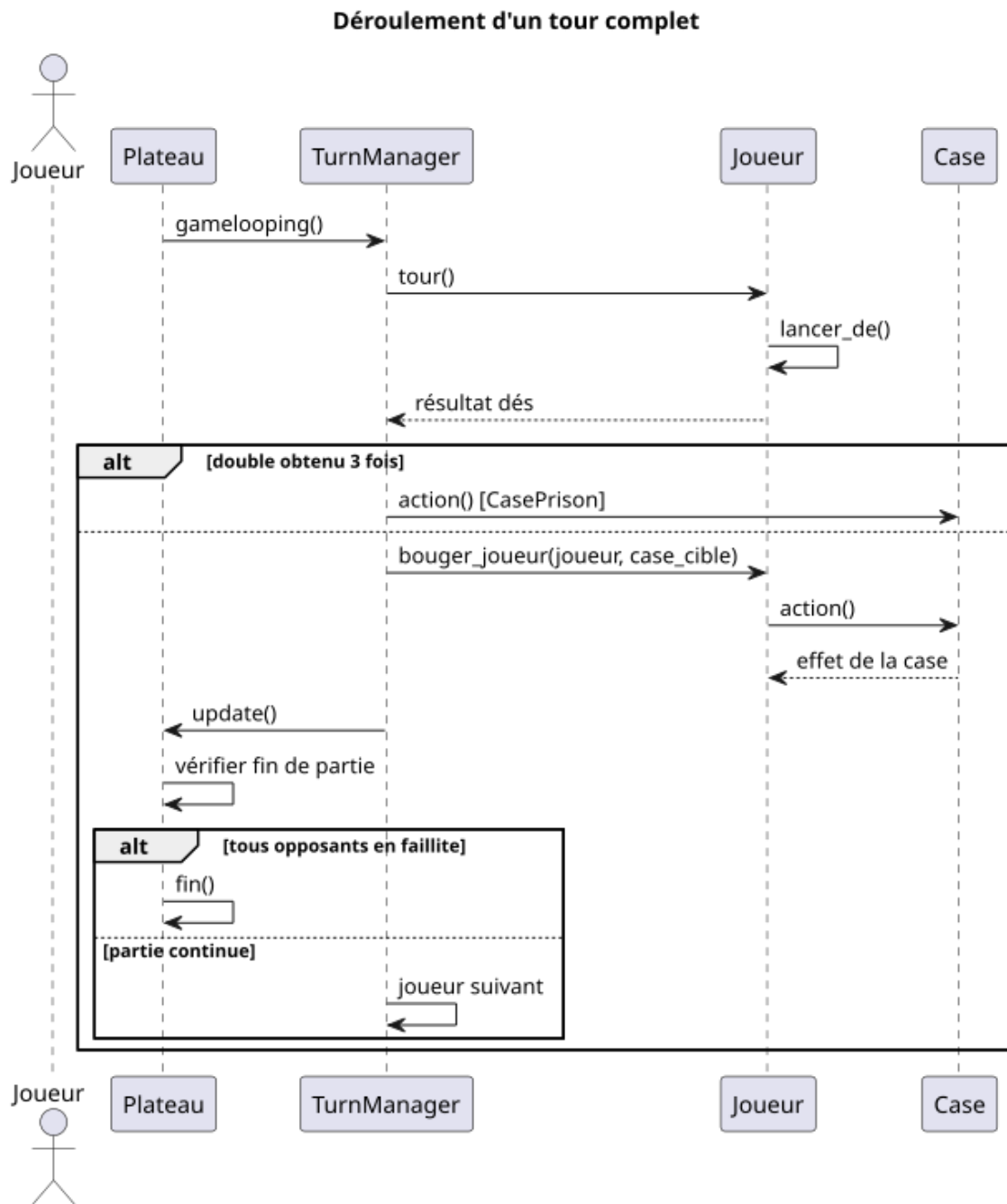


FIGURE 3 – Diagramme de séquence — Déroulement d'un tour complet. Le **Plateau** déclenche la boucle de jeu via `gamelooping()`, qui délègue au **TurnManager** l'appel à `tour()` sur le **Joueur** courant. Après le lancer de dés, deux branches sont possibles : si le joueur obtient un double trois fois de suite il est envoyé directement en prison ; sinon il est déplacé sur la case cible dont l'`action()` est exécutée. En fin de tour le plateau vérifie les conditions de victoire.

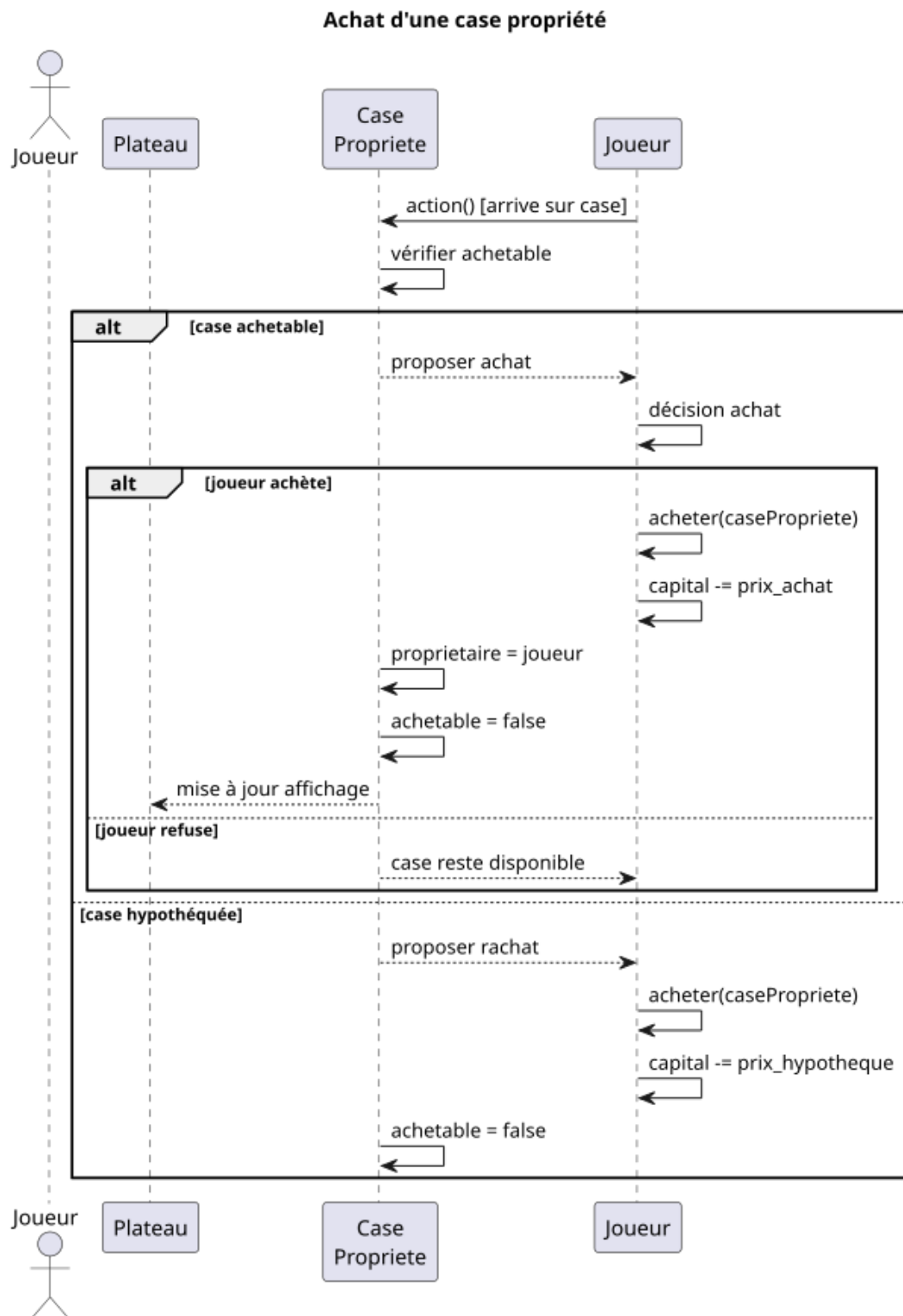


FIGURE 4 – Diagramme de séquence — Achat d'une case propriété. Lorsqu'un joueur arrive sur une **CasePropriete**, trois situations sont traitées : la case est achetable (le joueur peut l'acquérir au **prix_achat**), le joueur refuse l'achat (la case reste disponible), ou la case est hypothéquée (rachat au **prix_hypothèque**). Dans les deux cas d'achat, l'attribut **achetable** est mis à **false** et le **proprietaire** est mis à jour.

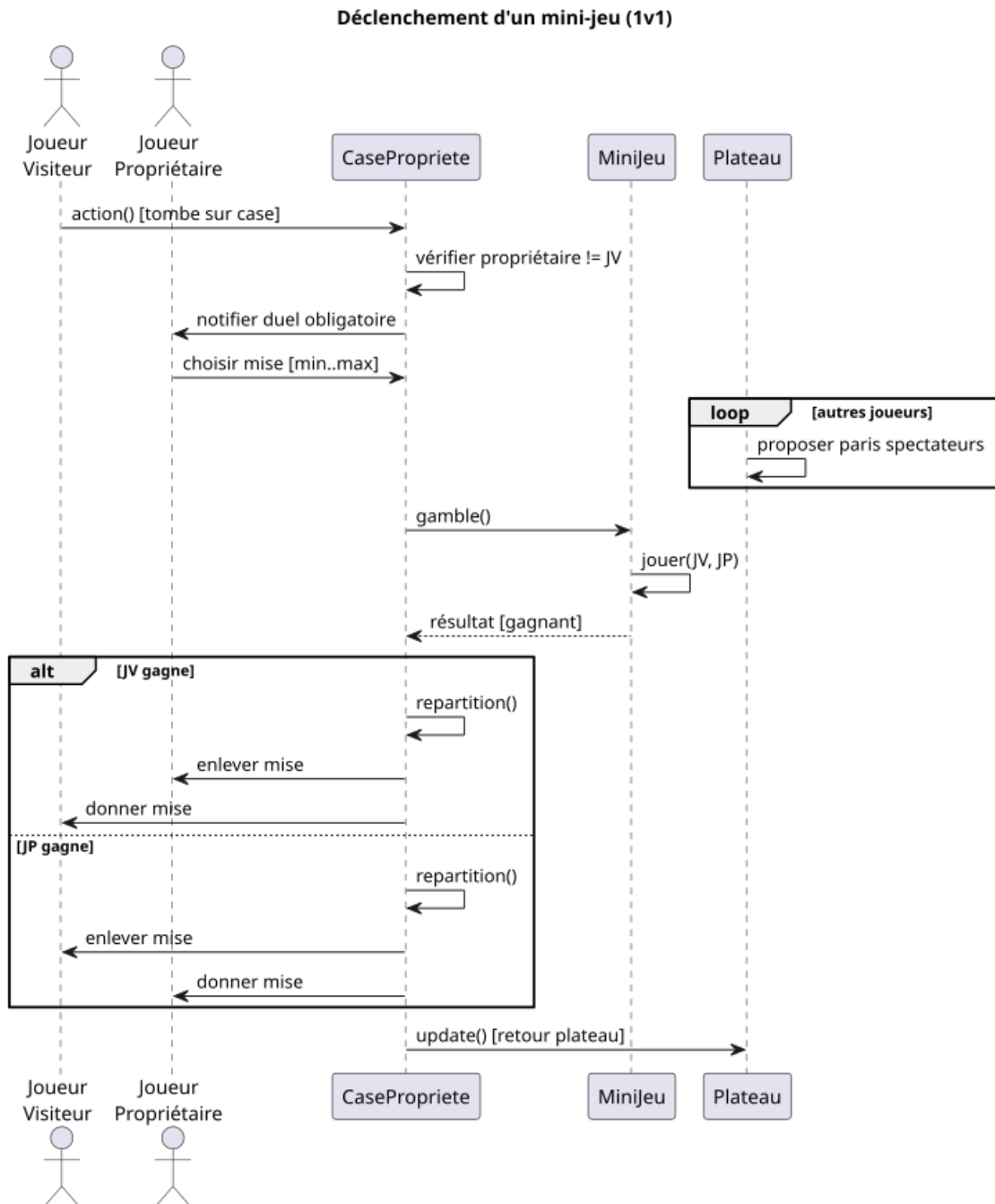


FIGURE 5 – Diagramme de séquence — Déclenchement d'un mini-jeu (1v1). Quand le joueur visiteur atterrit sur une **CasePropriete** appartenant à un autre joueur, un duel est obligatoirement déclenché. Le propriétaire fixe la mise dans l'intervalle autorisé, les autres joueurs peuvent parier, puis **gamble()** lance le **MiniJeu**. À l'issue du duel, **repartition()** transfère la mise du perdant vers le gagnant avant de restituer la main au plateau.

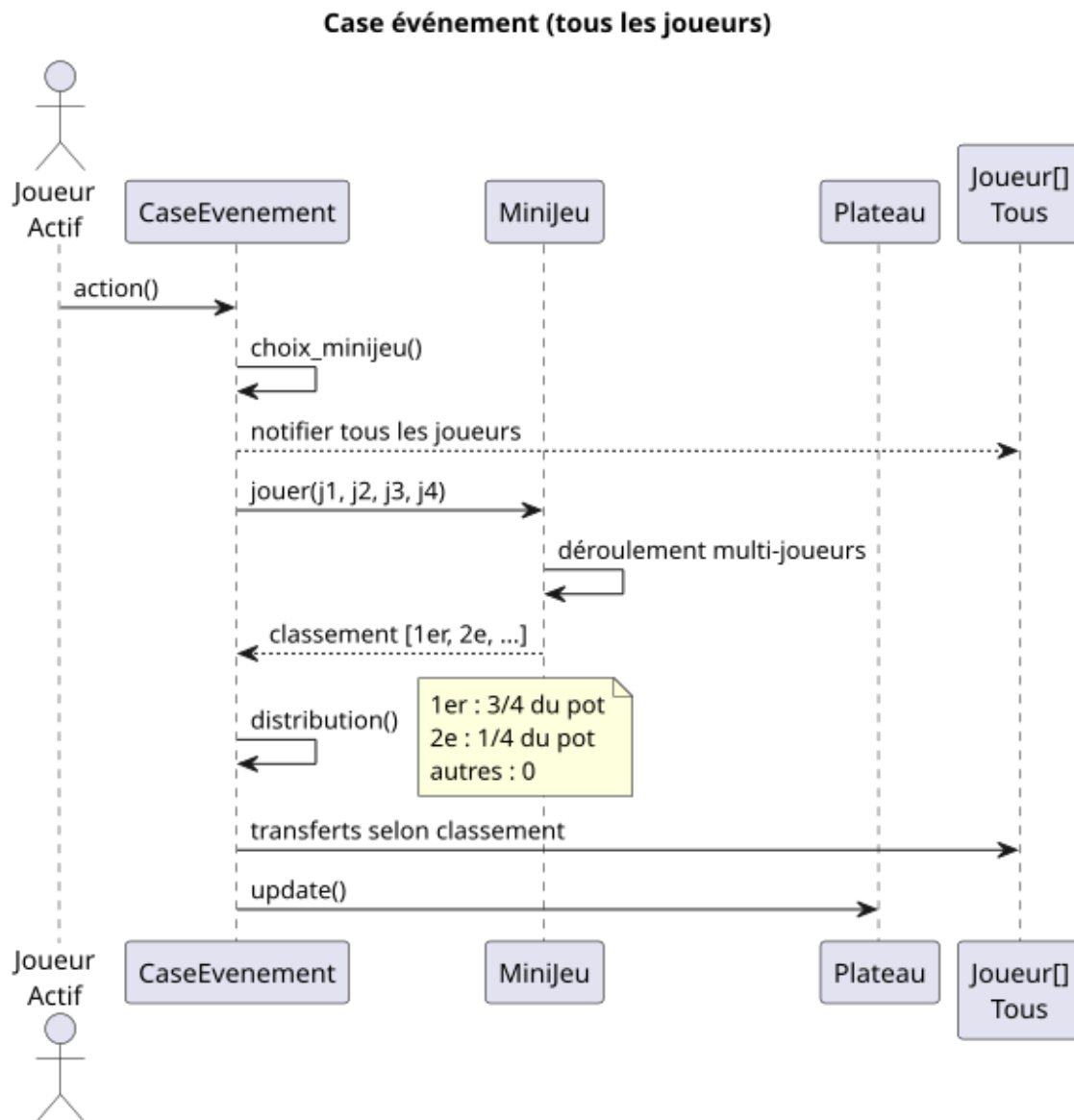


FIGURE 6 – Diagramme de séquence — Case événement (tous les joueurs). Lorsqu'un joueur tombe sur une *CaseEvenement*, *choix_minijeu()* sélectionne le mini-jeu associé et tous les joueurs sont notifiés. Le *MiniJeu* est joué par l'ensemble des participants; à la fin, *distribution()* applique la règle de répartition : le premier reçoit les $\frac{3}{4}$ du pot, le second le $\frac{1}{4}$ restant, les autres ne reçoivent rien.

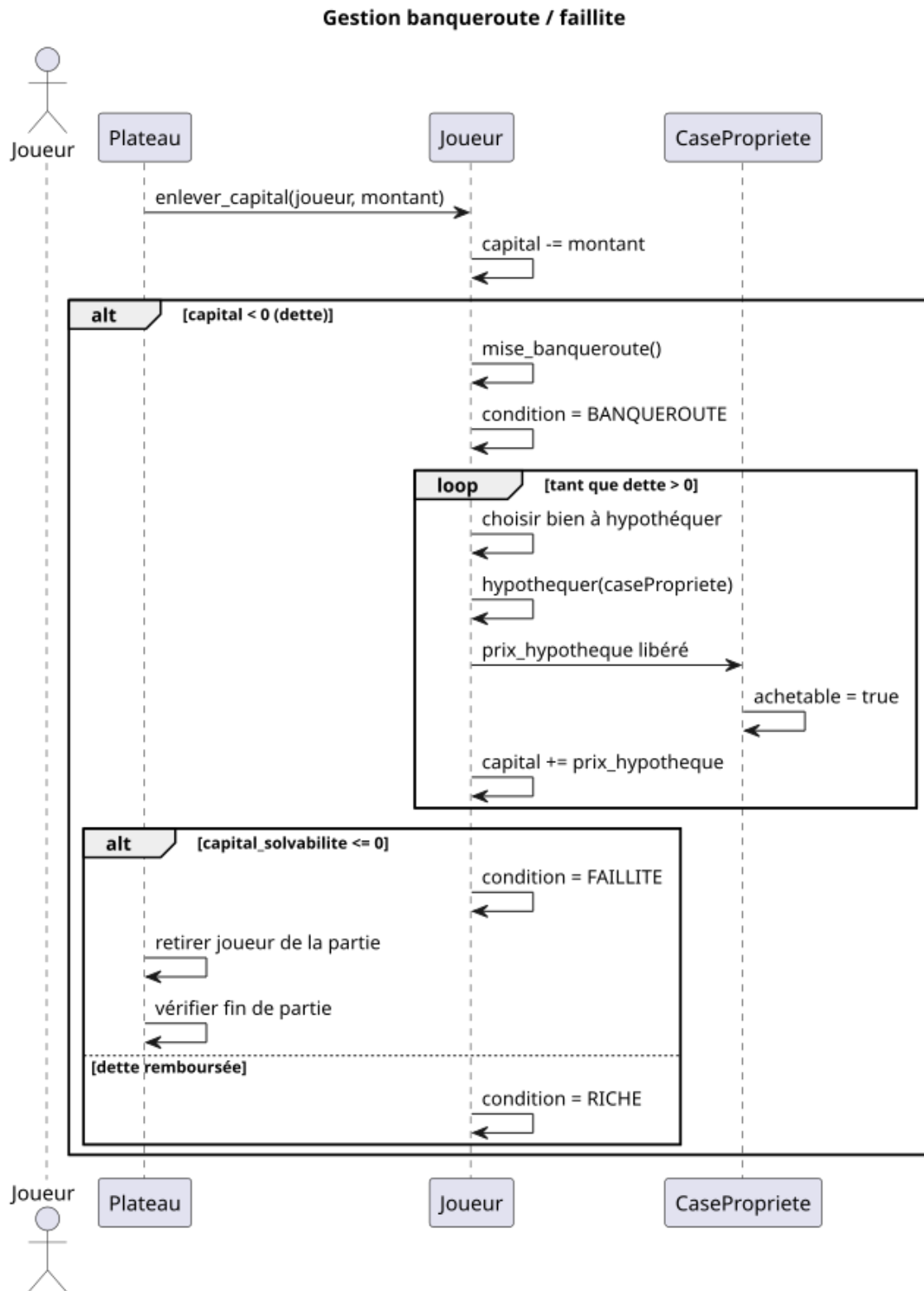


FIGURE 7 – Diagramme de séquence — Gestion banqueroute / faillite. Quand `enlever_capital()` fait passer le solde en dessous de zéro, le joueur entre en état **BANQUEROUTE** et doit rembourser sa dette en hypothéquant ses **CasePropriete** (boucle *tant que dette > 0*). Si le capital de solvabilité atteint zéro malgré tout, l'état passe à **FAILLITE** : le joueur est retiré de la partie et la condition de victoire est vérifiée. Dans le cas contraire, son état repasse à **RICHE**.

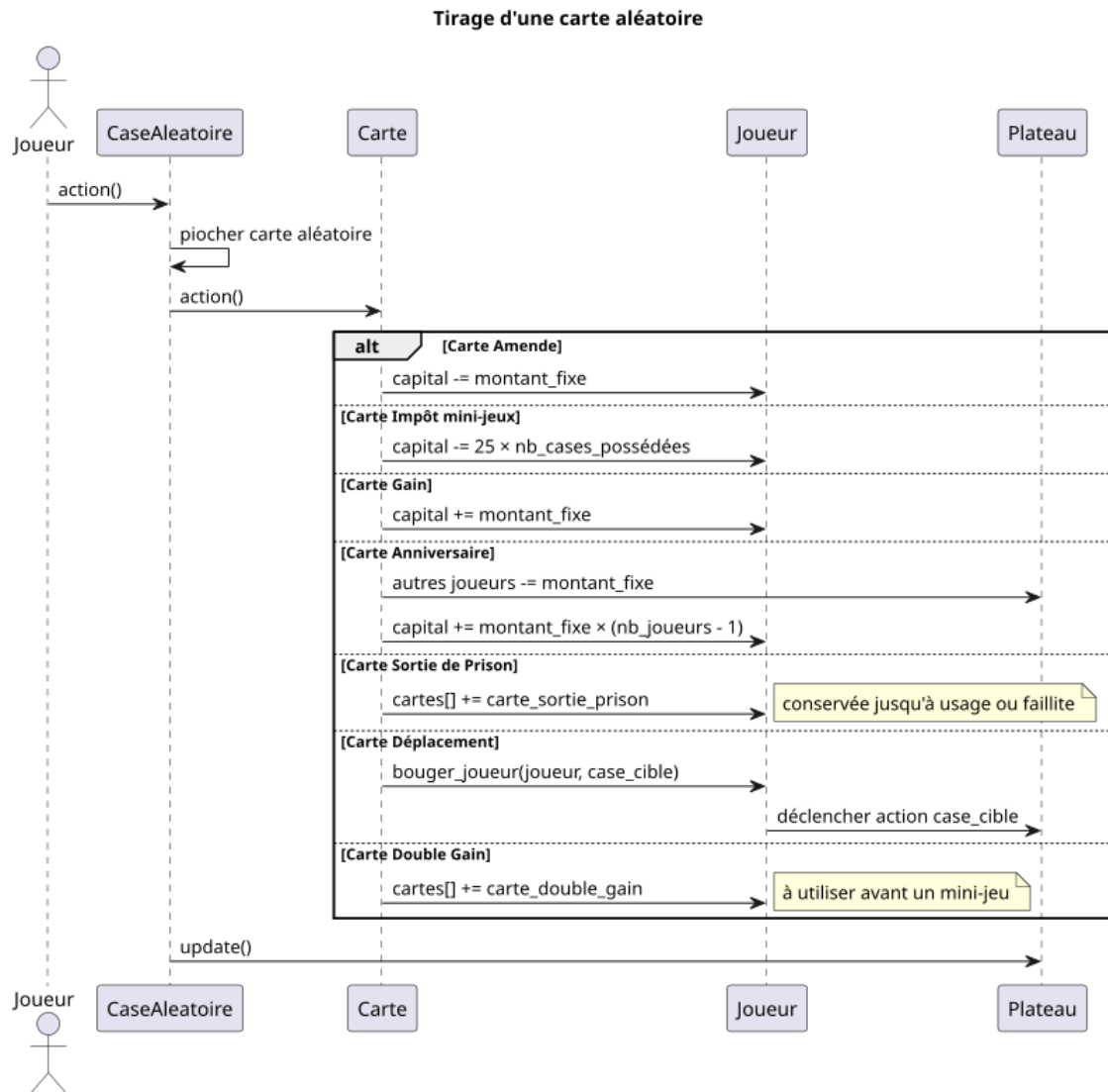


FIGURE 8 – Diagramme de séquence — Tirage d'une carte aléatoire. Lorsqu'un joueur tombe sur une *CaseAleatoire*, une *Carte* est piochée et son *action()* est exécutée. Sept effets sont possibles : *Amende* (paiement fixe), *Impôt mini-jeux* (paiement proportionnel aux cases possédées : $25 \times x$), *Gain* (encaissement fixe), *Anniversaire* (les autres joueurs versent une somme fixe), *Sortie de prison* (carte conservée jusqu'à usage ou faillite), *Déplacement* (téléportation sur une case cible) et *Double Gain* (multiplie les gains du prochain mini-jeu remporté).

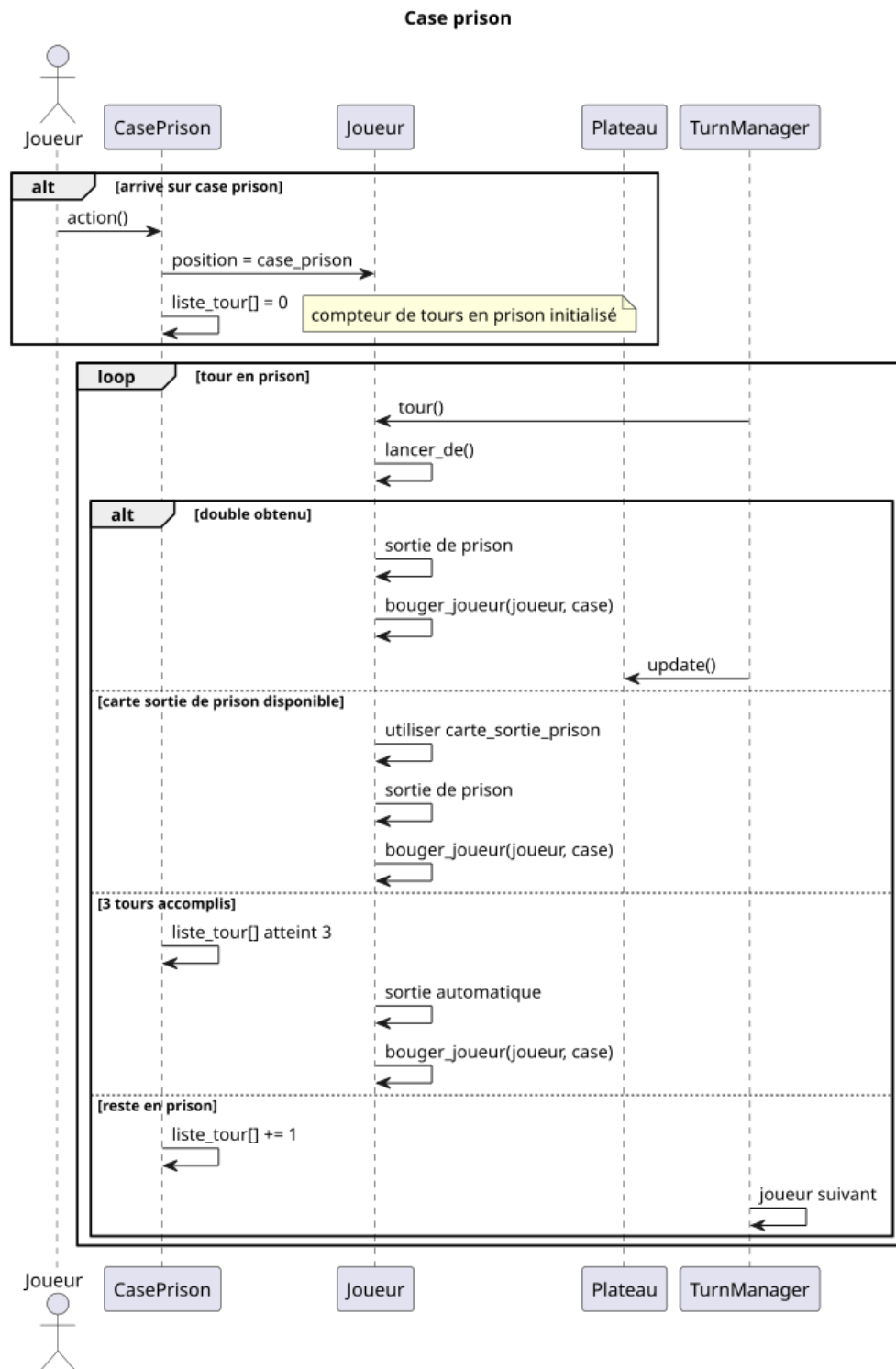


FIGURE 9 – Diagramme de séquence — Case prison. À l'arrivée en prison, la position du joueur est fixée et le compteur `liste_tour[]` est initialisé à zéro. À chaque tour en prison, trois sorties sont possibles : obtenir un double aux dés, utiliser une *carte sortie de prison*, ou avoir accompli trois tours complets (sortie automatique). Dans tous les autres cas, `liste_tour[]` est incrémenté et le `TurnManager` passe au joueur suivant.

4 Tests d'intégrations

Voici la liste exhaustive des tests d'intégrations

4.1 Gestion de la partie (10 tests)

TI-P01 - Initialisation d'une partie (2 à 4 joueurs)

Entrées	Nombre de joueurs : 2 à 4 — Noms des joueurs — Temps de jeu (en minutes)
Étapes	1. Appeler nb_joueur() → saisie du nombre de joueurs 2. Appeler nom_joueurs() → saisie des noms 3. Appeler ordre() → lancer_de() pour chaque joueur, réorganisation du tableau joueurs[] 4. Appeler temps_jeu() → saisie de la durée de partie 5. Vérifier que tous les joueurs ont capital = capital_initial, position = case 0, proprietes[] vide
Composants	Plateau, Joueur, TurnManager
Diagramme réf.	Figure 3 - Déroulement d'un tour complet
Résultat attendu	joueurs[] trié du plus grand jet au plus petit ; chaque Joueur : capital = capital_initial (calculé selon nb_joueurs et temps_jeu), position = case 0

TI-P02 - Capital initial dépend du nombre de joueurs et du temps de jeu

Entrées	Deux parties : (2 joueurs, 30 min) puis (4 joueurs, 90 min)
Étapes	1. Lancer la partie 1 (2 joueurs, 30 min) → relever capital_initial 2. Lancer la partie 2 (4 joueurs, 90 min) → relever capital_initial 3. Comparer les deux valeurs
Composants	Plateau, Joueur
Diagramme réf.	Figure 3 - Déroulement d'un tour complet
Résultat attendu	capital_initial(2j, 30min) ≠ capital_initial(4j, 90min) ; au sein d'une même partie tous les joueurs ont le même capital_initial

TI-P03 - Ordre des joueurs établi au premier tour par les dés

Entrées	3 joueurs en cours d'initialisation — Jets : J1=8, J2=11, J3=5
Étapes	1. Appeler ordre() → lancer_de() x 3 2. Enregistrer les scores : J1=8, J2=11, J3=5 3. Réorganiser joueurs[] → [J2, J1, J3] 4. Vérifier que l'ordre est maintenu pour toute la partie
Composants	Plateau, Joueur, TurnManager
Diagramme réf.	Figure 3 - Déroulement d'un tour complet
Résultat attendu	joueurs[] = [J2, J1, J3] ; ordre maintenu pour toute la partie

TI-P04 - Égalité au jet initial : relance pour départager

Entrées	2 joueurs — J1=7, J2=7 au premier jet
Étapes	1. ordre() → J1=7, J2=7 → égalité détectée 2. Relance pour les deux joueurs concernés : J1=9, J2=4 3. joueurs[] = [J1, J2]
Composants	Plateau, Joueur
Diagramme réf.	Figure 3 - Déroulement d'un tour complet
Résultat attendu	Relance effectuée sans erreur ; joueurs[] = [J1, J2] ; ordre cohérent

TI-P05 - Lancer de dés et déplacement normal

Entrées	Joueur A à la case 5 — Jet = 3+4=7 (pas de double)
Étapes	1. Appeler tour() → lancer_de() x 2 → total = 7 2. Appeler bouger_joueur(case 12) 3. Appeler action() sur la case 12 4. Appeler update() → vérifier passage au joueur suivant
Composants	Plateau, Joueur, Case
Diagramme réf.	Figure 3 - Déroulement d'un tour complet
Résultat attendu	position(A) = 12 ; action() de la case 12 exécutée ; tour passé au joueur suivant

TI-P06 - Double aux dés : le joueur rejoue

Entrées	Joueur A à la case 3 — compteur_double = 0 — Jet 1 : 3+3 (double) — Jet 2 : 2+5
Étapes	1. tour() → lancer_de() = 3+3 → double détecté → compteur_double = 1 2. bouger_joueur(case 9) 3. Rejouer : lancer_de() = 2+5 → bouger_joueur(case 16) 4. Fin du tour → compteur_double = 0
Composants	Plateau, Joueur
Diagramme réf.	Figure 3 - Déroulement d'un tour complet
Résultat attendu	A s'est déplacé deux fois dans le même tour ; compteur_double = 0 en fin de tour

TI-P07 - Deux doubles consécutifs : rejoue une deuxième fois

Entrées	Joueur A — compteur_double = 0 — Jet 1 : 4+4 — Jet 2 : 2+2 — Jet 3 : 1+5
Étapes	1. tour() → 4+4 → compteur_double = 1 → rejoue 2. 2+2 → compteur_double = 2 → rejoue 3. 1+5 → déplacement normal → fin du tour → compteur_double = 0
Composants	Plateau, Joueur
Diagramme réf.	Figure 3 - Déroulement d'un tour complet
Résultat attendu	A a joué 3 fois dans le même tour ; aucun envoi en prison ; compteur_double = 0

TI-P08 - Triple double envoi en prison

Entrées	Joueur A — compteur_double = 2 — prochain jet = double (3+3)
Étapes	1. tour() → lancer_de() = 3+3 → compteur_double = 3 2. Envoi direct → CasePrison.action() appelé 3. position(A) = case_prison, liste_tour[A] = 0, compteur_double = 0 4. Aucune somme de case départ créditée
Composants	Plateau, Joueur, CasePrison
Diagramme réf.	Figure 3 - Déroulement d'un tour complet
Résultat attendu	A en prison ; liste_tour[A] = 0 ; compteur_double = 0 ; pas de bonus case départ

TI-P09 - Fin de partie : tous les adversaires en faillite

Entrées	Partie à 3 joueurs — J2 et J3 viennent de tomber en faillite
Étapes	1. J3 déclaré FAILLITE → retiré de joueurs[] 2. Plateau vérifie fin_de_partie() : nb joueurs actifs = 1 3. fin() appelé → J1 annoncé vainqueur
Composants	Plateau, Joueur, TurnManager
Diagramme réf.	Figure 3 - Déroulement d'un tour complet
Résultat attendu	Partie stoppée ; J1 déclaré vainqueur ; plus aucun tour exécuté

TI-P10 - Arrêt manuel → vainqueur au patrimoine le plus élevé

Entrées	Partie à 4 joueurs actifs — Arrêt manuel déclenché — J1 : capital=200 + val=100 — J2 : capital=450
Étapes	1. fin() appelé (arrêt manuel) 2. Pour chaque joueur : patrimoine = capital + val_proprietes 3. J1 = 300 €, J2 = 450 €... → tri décroissant 4. Annonce du classement
Composants	Plateau, Joueur, CasePropriete
Diagramme réf.	Figure 3 - Déroulement d'un tour complet
Résultat attendu	J2 vainqueur (450 € > 300 €) ; classement affiché sans erreur

4.2 Gestion des joueurs (8 tests)

TI-J01 - Déplacement normal : position et action de la case

Entrées	Joueur A à la case 5 — lancer_de() x 2 = 7
Étapes	1. tour() → lancer_de() x 2 = 7 2. bouger_joueur(A, case 12) appelé 3. case[12].action() exécuté
Composants	Joueur, Plateau, Case
Diagramme réf.	Figure 3 - Déroulement d'un tour complet
Résultat attendu	position(A) = 12 (5+7) ; action() de la case 12 déclenchée

TI-J02 - Passage par la case départ : somme fixe créditée

Entrées	Joueur A à la case 38 — CaseDepart.montant = 200 — jet = 4
Étapes	1. bouger_joueur(A, case 42 modulo case 2) → passe par case 0 2. CaseDepart.action() appelé 3. donner_capital(A, 200) appelé dans Plateau
Composants	CaseDepart, Joueur, Plateau
Diagramme réf.	Figure 3 - Déroulement d'un tour complet
Résultat attendu	capital(A) = ancien + 200 ; somme créditée exactement une fois

TI-J03 - Tour complet sans case spéciale : somme créditée une seule fois

Entrées	Joueur A effectue un tour complet — CaseDepart.montant = 200
Étapes	1. A parcourt toutes les cases sans mini-jeu ni carte 2. Passage unique par la case 0 → CaseDepart.action() 3. Vérifier que donner_capital() n'est appelé qu'une fois
Composants	CaseDepart, Joueur, Plateau
Diagramme réf.	Figure 3 - Déroulement d'un tour complet
Résultat attendu	capital(A) = ancien + 200 ; donner_capital() appelé exactement 1 fois

TI-J04 - Triple double → prison sans passer par la case départ

Entrées	Joueur A — compteur_double = 0 — 3 jets doubles consécutifs
Étapes	1. tour() → double 1 : compteur_double = 1 2. double 2 : compteur_double = 2 3. double 3 : compteur_double = 3 → CasePrison.action() 4. position = case_prison, liste_tour[A] = 0, compteur_double = 0 5. Vérifier : donner_capital() NON appelé (case départ non franchise)
Composants	Joueur, CasePrison, Plateau
Diagramme réf.	Figure 3 - Déroulement d'un tour complet
Résultat attendu	A en prison ; case départ non franchise ; compteur_double = 0

TI-J05 - Joueur IA : déplacement et décision d'achat automatiques

Entrées	Partie avec 1 joueur humain et 1 IA — Tour de l'IA — case achetable atteinte
Étapes	1. tour() appelé sur l'IA → lancer_de() automatique 2. bouger_joueur(IA, case_cible) 3. CasePropriete.action() → achetable = true → IA prend la décision sans input humain 4. Si décision d'achat : acheter(casePropriete) appelé
Composants	Joueur (IA), Plateau, CasePropriete
Diagramme réf.	Figure 4 - Achat d'une case propriété
Résultat attendu	Tour de l'IA effectué sans blocage ; décision cohérente avec capital (IA) ; capital mis à jour si achat

TI-J06 - Joueur IA : décision de mise dans les bornes [min, max]

Entrées	IA propriétaire d'une case Pong (1 case possédée) — Joueur humain (JV) tombe dessus — bornes [50, 200]
Étapes	1. CasePropriete.action() → propriétaire = IA ≠ JV 2. IA choisit automatiquement une mise dans [50, 200] 3. gamble() appelé → MiniJeu.jouer(JV, IA) 4. Vérifier mise dans [min_actuel, max_actuel]
Composants	Joueur (IA), CasePropriete, MiniJeu
Diagramme réf.	Figure 5 - Déclenchement d'un mini-jeu (1v1)
Résultat attendu	mise choisie ∈ [50, 200] ; mini-jeu lancé sans blocage

TI-J07 - Calcul du capital de solvabilité

Entrées	Joueur A : capital = 150 € — 2 cases possédées (val.hypothèque = 100 € chacune)
Étapes	1. condition_financiere() appelé sur A 2. capital_solvabilite = capital + val_proprietes = 150 + 200 = 350 3. Vérifier la valeur retournée
Composants	Joueur, Plateau
Diagramme réf.	Figure 7 - Gestion banqueroute / faillite
Résultat attendu	condition_financiere() retourne capital_solvabilite = 350 €

TI-J08 - Faillite : retrait correct du joueur du tour

Entrées	Partie à 3 joueurs — A vient d'être déclaré en faillite
Étapes	1. condition(A) = FAILLITE 2. A retiré de joueurs[] dans Plateau 3. update() → vérifie fin de partie 4. Tours suivants : seuls B et C jouent
Composants	Joueur, Plateau, TurnManager
Diagramme réf.	Figure 7 - Gestion banqueroute / faillite
Résultat attendu	A absent de joueurs[] ; nb joueurs actifs = 2 ; partie continue entre B et C

4.3 Gestion économique (20 tests)**TI-E01 - Achat d'une case propriété libre**

Entrées	Joueur A : capital = 500 € — CasePropriete : achetable = true, prix_achat = 200
Étapes	1. bouger_joueur(A, casePropriete) 2. CasePropriete.action() → achetable = true → propose achat 3. A confirme → acheter(casePropriete) appelé 4. capital -= prix_achat ; proprietes[] += case ; val_proprietes += prix_achat 5. achetable = false ; propriétaire = A
Composants	CasePropriete, Joueur, Plateau
Diagramme réf.	Figure 4 - Achat d'une case propriété
Résultat attendu	capital(A) = 300 € ; achetable = false ; propriétaire = A ; case dans proprietes[A]

TI-E02 - Tentative d'achat sans fonds suffisants : achat refusé

Entrées	Joueur A : capital = 100 € — CasePropriete : prix_achat = 200
Étapes	1. bouger_joueur(A, casePropriete) 2. CasePropriete.action() → achetable = true 3. Vérification capital : 100 ; 200 → achat impossible 4. Afficher message d'erreur
Composants	CasePropriete, Joueur, Plateau
Diagramme réf.	Figure 4 - Achat d'une case propriété
Résultat attendu	achetable = true (inchangé) ; propriétaire = null ; capital(A) = 100 € (inchangé)

TI-E03 - Refus d'achat : case reste disponible

Entrées	Joueur A solvable — CasePropriete achetable = true
Étapes	1. bouger_joueur(A, casePropriete) 2. action() propose achat → A refuse 3. Vérifier achetable = true, propriétaire = null, capital inchangé
Composants	CasePropriete, Joueur, Plateau
Diagramme réf.	Figure 4 - Achat d'une case propriété
Résultat attendu	achetable = true ; capital(A) inchangé ; propriétaire = null

TI-E04 - Rachat d'une case hypothéquée par son propriétaire

Entrées	Joueur A : capital = 200 € — CasePropriete hypothéquée (achetable = true, val_hypothèque = 100)
Étapes	1. bouger_joueur(A, casePropriete) → case hypothéquée 2. action() → propose rachat 3. A confirme acheter(casePropriete) 4. capital -= prix_hypothèque ; achetable = false ; propriétaire = A
Composants	CasePropriete, Joueur, Plateau
Diagramme réf.	Figure 4 - Achat d'une case propriété
Résultat attendu	capital(A) = 100 € ; achetable = false ; propriétaire = A

TI-E05 - Achat d'une case hypothéquée par un autre joueur

Entrées	Case de A hypothéquée (achetable = true) — B atterrit dessus — prix_hypothèque = 75
Étapes	1. bouger_joueur(B, casePropriete) 2. action() détecte case hypothéquée → propose rachat à B 3. B confirme acheter(casePropriete) 4. capital(B) -= 75 ; achetable = false ; proprietaire = B
Composants	CasePropriete, Joueur, Plateau
Diagramme réf.	Figure 4 - Achat d'une case propriété
Résultat attendu	capital(B) = ancien - 75 ; proprietaire = B ; achetable = false

TI-E06 - Mise en hypothèque d'un bien

Entrées	Joueur A : capital = 50 € — CasePropriete possédée (val_hypothèque = 100)
Étapes	1. A appelle hypothéquer(casePropriete) 2. capital += val_hypothèque → capital = 150 3. achetable = true (case marquée hypothéquée) 4. case retirée de proprietes[A] (ou flaggée)
Composants	Joueur, CasePropriete, Plateau
Diagramme réf.	Figure 7 - Gestion banqueroute / faillite
Résultat attendu	capital(A) = 150 € ; achetable = true ; val_proprietes(A) réduit de 100

TI-E07 - Système de mise : bornes min et max respectées

Entrées	Propriétaire B : 1 case Pong — bornes [50, 200] — B propose 300 €
Étapes	1. gamble() → B choisit mise = 300 € 2. Système détecte 300 > max(200) 3. Mise rejetée ou ramenée à 200 4. Vérifier mise_effective ∈ [50, 200]
Composants	CasePropriete, Joueur, Plateau
Diagramme réf.	Figure 5 - Déclenchement d'un mini-jeu (1v1)
Résultat attendu	mise_effective ≤ 200 ; jeu lancé normalement

TI-E08 - Borne maximale augmente avec le nombre de cases du même type

Entrées	B : 1 case Pong (max = 200 €) — B achète une 2e case Pong
Étapes	1. Vérifier max_mise avec 1 case Pong = 200 2. B achète 2e case Pong → val_proprietes mis à jour 3. Vérifier max_mise avec 2 cases Pong > 200
Composants	CasePropriete, Joueur, Plateau
Diagramme réf.	Figure 5 - Déclenchement d'un mini-jeu (1v1)
Résultat attendu	max_mise(2 cases Pong) > max_mise(1 case Pong)

TI-E09 - Pari spectateur $\leq$ capital de solvabilité : accepté

Entrées	Spectateur C : capital = 100 €, val_proprietes = 50 € $\rightarrow$ solvabilité = 150 € — Pari = 150 €
Étapes	1. C saisit pari = 150 € 2. Vérification : $150 \leq \text{capital_solvabilite}(C) \rightarrow \text{OK}$ 3. Pari enregistré
Composants	Joueur, Plateau
Diagramme réf.	Figure 5 - Déclenchement d'un mini-jeu (1v1)
Résultat attendu	Pari de 150 € enregistré ; capital(C) gelé pendant le mini-jeu

TI-E10 - Pari spectateur $\nless$ capital de solvabilité : refusé

Entrées	Spectateur C : solvabilité = 150 € — Pari tenté = 200 €
Étapes	1. C saisit pari = 200 € 2. Vérification : $200 \nless \text{capital_solvabilite}(C) \rightarrow \text{refus}$ 3. Pari non enregistré
Composants	Joueur, Plateau
Diagramme réf.	Figure 5 - Déclenchement d'un mini-jeu (1v1)
Résultat attendu	Pari refusé ; capital(C) inchangé

TI-E11 - Redistribution après mini-jeu 1v1 : le gagnant récupère le pot

Entrées	Duel A (JV) vs B (JP) — mise = 100 € — C a parié 50 € sur A — A gagne
Étapes	1. Mini-jeu terminé $\rightarrow$ résultat [gagnant = A] 2. repartition() appelé 3. enlever_capital(B, 100) ; donner_capital(A, 100) 4. Gérer le pari de C selon le résultat
Composants	CasePropriete, MiniJeu, Joueur, Plateau
Diagramme réf.	Figure 5 - Déclenchement d'un mini-jeu (1v1)
Résultat attendu	capital(A) += 100 ; capital(B) -= 100 ; pari de C réglé (gains versés)

TI-E12 - Redistribution après jeu événement

Entrées	4 joueurs — case événement — pot = 200 € — Classement : J1, J2, J3, J4
Étapes	1. distribution() appelé 2. J1 reçoit $3/4 \times 200 = 150$ € $\rightarrow$ donner_capital(J1, 150) 3. J2 reçoit $1/4 \times 200 = 50$ € $\rightarrow$ donner_capital(J2, 50) 4. J3, J4 : rien
Composants	CaseEvenement, MiniJeu, Joueur, Plateau
Diagramme réf.	Figure 6 - Case événement (tous les joueurs)
Résultat attendu	capital(J1) += 150 ; capital(J2) += 50 ; J3/J4 inchangés ; total redistribué = 200 €

TI-E13 - Redistribution des paris spectateurs après résultat

Entrées	C a parié 80 € sur A — D a parié 60 € sur B — A gagne
Étapes	1. Résultat transmis : gagnant = A 2. C (pari gagnant) → gain crédité 3. D (pari perdant) → enlever_capital(D, 60)
Composants	Joueur, Plateau
Diagramme réf.	Figure 5 - Déclenchement d'un mini-jeu (1v1)
Résultat attendu	capital(D) -= 60; capital(C) += gain selon cote; total des paris conservé

TI-E14 - Entrée en banqueroute

Entrées	Joueur A : capital = 30 € — Mise à payer = 100 € (A perd)
Étapes	1. enlever_capital(A, 100) → capital = 30 - 100 = -70 2. capital < 0 → mise.banqueroute() appelé 3. condition(A) = BANQUEROUTE; dette = 70 €; capital = 0
Composants	Joueur, Plateau
Diagramme réf.	Figure 7 - Gestion banqueroute / faillite
Résultat attendu	condition(A) = BANQUEROUTE; capital(A) = 0; dette = 70 €

TI-E15 - Banqueroute : remboursement par hypothèque de biens

Entrées	A en BANQUEROUTE — dette = 100 € — 2 cases possédées (val.hypothèque = 60 € chacune)
Étapes	1. mise.banqueroute() → boucle tant que dette > 0 2. A choisit case 1 → hypothéquer(case1) → capital += 60 3. dette = 100 - 60 = 40 > 0 → A choisit case 2 → hypothéquer(case2) → capital += 60 4. dette = 40 - 60 < 0 → dette = 0 → condition(A) = RICHE
Composants	Joueur, CasePropriete, Plateau
Diagramme réf.	Figure 7 - Gestion banqueroute / faillite
Résultat attendu	condition(A) = RICHE; capital(A) = 20 €; 2 cases marquées achetable = true

TI-E16 - Faillite déclarée quand capital de solvabilité atteint zéro

Entrées	A en BANQUEROUTE — dette = 200 € — aucun bien hypothécable
Étapes	1. mise.banqueroute() → aucun bien 2. condition(A) = FAILLITE 3. A perd ses propriétés et son capital 4. A retiré de joueurs[]; Plateau vérifie fin de partie
Composants	Joueur, Plateau
Diagramme réf.	Figure 7 - Gestion banqueroute / faillite
Résultat attendu	condition(A) = FAILLITE; joueurs[].size() -= 1; propriétés libérées

TI-E17 - Transfert via banque artificielle (carte gain)

Entrées	Joueur A tire une carte gain — montant_fixe = 100 €
Étapes	1. CaseAleatoire.action() → carte piochée = Carte Gain 2. Carte.action() → donner_capital(A, 100) 3. Vérifier capital(A)
Composants	Carte, Joueur, Plateau
Diagramme réf.	Figure 8 - Tirage d'une carte aléatoire
Résultat attendu	capital(A) = ancien + 100 ; masse monétaire totale augmente

TI-E18 - Transfert via banque artificielle (carte amende)

Entrées	Joueur A tire une carte amende — montant_fixe = 80 € — capital(A) = 300 €
Étapes	1. Carte Amende piochée 2. Carte.action() → enlever_capital(A, 80) 3. Vérifier capital(A)
Composants	Carte, Joueur, Plateau
Diagramme réf.	Figure 8 - Tirage d'une carte aléatoire
Résultat attendu	capital(A) = 220 € ; masse monétaire totale diminue

TI-E19 - Transfert via pot commun : cohérence des soldes

Entrées	Mini-jeu 1v1 — mise(A) = 100 €, mise(B) = 100 € → pot = 200 € — A gagne
Étapes	1. enlever_capital(A, 100) ; enlever_capital(B, 100) → pot = 200 2. A gagne → donner_capital(A, 200) 3. Vérifier capitaux
Composants	MiniJeu, Joueur, Plateau
Diagramme réf.	Figure 5 - Déclenchement d'un mini-jeu (1v1)
Résultat attendu	capital(A) net = +100 € ; capital(B) net = -100 € ; somme totale conservée

TI-E20 - Transfert de joueur à joueur (carte anniversaire)

Entrées	4 joueurs, tous à 500 € — Carte anniversaire tirée par A — montant_fixe = 50 €
Étapes	1. Carte.action() → pour chaque autre joueur : transfert_argent(joueur, A, 50) 2. B, C, D : enlever_capital(X, 50) x 3 3. donner_capital(A, 150) = 50 x 3
Composants	Carte, Joueur, Plateau
Diagramme réf.	Figure 8 - Tirage d'une carte aléatoire
Résultat attendu	capital(A) = 650 € ; capital(B)=capital(C)=capital(D) = 450 €

4.4 Gestion du plateau (23 tests)

TI-B01 - Arrivée sur case mini-jeu libre : proposition d'achat

Entrées	CasePropriete : achetable = true, propriétaire = null — Joueur A atterrit dessus
Étapes	1. bouger_joueur(A, casePropriete) 2. CasePropriete.action() → vérifier achetable = true 3. Proposition d'achat affichée à A 4. Vérifier qu'aucun MiniJeu n'est instancié
Composants	CasePropriete, Joueur
Diagramme réf.	Figure 4 - Achat d'une case propriété
Résultat attendu	Proposition d'achat affichée ; MiniJeu NON lancé ; achetable = true

TI-B02 - Arrivée sur sa propre case : aucune action déclenchée

Entrées	CasePropriete : propriétaire = A — A atterrit dessus
Étapes	1. bouger_joueur(A, casePropriete) 2. CasePropriete.action() → vérifie propriétaire == A → condition vraie 3. Aucun mini-jeu, aucune proposition d'achat
Composants	CasePropriete, Joueur
Diagramme réf.	Figure 4 - Achat d'une case propriété
Résultat attendu	Tour continue normalement ; aucun MiniJeu instancié ; capital(A) inchangé

TI-B03 - Arrivée sur case possédée par un autre : mini-jeu déclenché

Entrées	CasePropriete : propriétaire = B — Joueur A (JV) atterrit dessus
Étapes	1. bouger_joueur(A, casePropriete) 2. CasePropriete.action() → propriétaire = B $\neq$ A → duel obligatoire 3. B notifié → choisit mise dans [min, max] 4. gamble() → MiniJeu.jouer(A, B)
Composants	CasePropriete, MiniJeu, Joueur
Diagramme réf.	Figure 5 - Déclenchement d'un mini-jeu (1v1)
Résultat attendu	Mini-jeu lancé entre A et B ; mise dans les bornes

TI-B04 - Arrivée sur case hypothéquée : proposition d'achat au prix_hypotheque

Entrées	CasePropriete hypothéquée : achetable = true — Joueur A atterrit dessus — prix_hypotheque = 75
Étapes	1. bouger_joueur(A, casePropriete) 2. CasePropriete.action() → achetable = true (case hypothéquée) 3. Proposition de rachat à 75 € affichée à A
Composants	CasePropriete, Joueur
Diagramme réf.	Figure 4 - Achat d'une case propriété
Résultat attendu	Proposition affichée à 75 € ; aucun mini-jeu ; si achat : capital(A) -= 75

TI-B05 - Arrivée sur case tirage de cartes : une carte tirée et son effet appliqué

Entrées	Joueur A atterrit sur CaseAleatoire
Étapes	1. bouger_joueur(A, caseAleatoire) 2. CaseAleatoire.action() → pioche première carte de la liste 3. Carte.action() exécuté → effet appliqué à A 4. Carte replacée en fin de liste
Composants	CaseAleatoire, Carte, Joueur, Plateau
Diagramme réf.	Figure 8 - Tirage d'une carte aléatoire
Résultat attendu	Effet de la carte appliqué à A ; une seule carte piochée par tour

TI-B06 - Carte amende : mise fixe prélevée

Entrées	Joueur A : capital = 300 € — Carte Amende piochée — montant_fixe = 80 €
Étapes	1. Carte.action() → enlever_capital(A, 80) 2. Vérifier capital(A)
Composants	Carte, Joueur, Plateau
Diagramme réf.	Figure 8 - Tirage d'une carte aléatoire
Résultat attendu	capital(A) = 220 € (300 - 80)

TI-B07 - Carte impôt sur mini-jeux : montant proportionnel aux possessions

Entrées	Joueur A : 3 cases possédées — Carte Impôt piochée — formule : 25 x nb_cases
Étapes	1. Carte.action() → x = nb cases possédées par A = 3 2. montant = 25 x 3 = 75 € 3. enlever_capital(A, 75)
Composants	Carte, Joueur, Plateau
Diagramme réf.	Figure 8 - Tirage d'une carte aléatoire
Résultat attendu	capital(A) -= 75 €

TI-B08 - Carte gain : somme créditée depuis la banque

Entrées	Joueur A — Carte Gain piochée — montant_fixe = 100 €
Étapes	1. Carte.action() → donner_capital(A, 100) 2. Vérifier capital(A)
Composants	Carte, Joueur, Plateau
Diagramme réf.	Figure 8 - Tirage d'une carte aléatoire
Résultat attendu	capital(A) = ancien + 100 €

TI-B09 - Carte anniversaire : chaque autre joueur paie une mise fixe

Entrées	4 joueurs, tous à 500 € — Carte Anniversaire piochée par A — montant_fixe = 50 €
Étapes	1. Carte.action() : pour chaque joueur $j \neq A$: transfert_argent(j, A, 50) 2. B, C, D : capital -= 50 x 3 ; A : capital += 150
Composants	Carte, Joueur, Plateau
Diagramme réf.	Figure 8 - Tirage d'une carte aléatoire
Résultat attendu	capital(A) = 650 € ; capital(B)=capital(C)=capital(D)=450 €

TI-B10 - Carte sortie de prison : conservée dans l'inventaire du joueur

Entrées	Joueur A — Carte Sortie de Prison piochée
Étapes	1. Carte.action() → cartes[A] += carte_sortie_prison 2. Aucun effet immédiat sur capital 3. Vérifier que la carte est dans cartes[A]
Composants	Carte, Joueur
Diagramme réf.	Figure 8 - Tirage d'une carte aléatoire
Résultat attendu	cartes[A] contient carte_sortie_prison ; capital(A) inchangé

TI-B11 - Carte sortie de prison utilisée lors d'un tour en prison

Entrées	A en prison — cartes[A] contient carte_sortie_prison — Début de tour
Étapes	1. CasePrison.action() → vérifie cartes[A] 2. A décide d'utiliser la carte 3. carte_sortie_prison retirée de cartes[A] 4. sortie de prison → bouger_joueur(A, case_cible)
Composants	CasePrison, Joueur, Carte
Diagramme réf.	Figure 9 - Case prison
Résultat attendu	A libéré ; carte absente de cartes[A] ; bouger_joueur() appelé normalement

TI-B12 - Carte sortie de prison disparaît lors de la faillite

Entrées	A déclaré en faillite — cartes[A] contient carte_sortie_prison
Étapes	1. condition(A) = FAILLITE 2. Toutes les propriétés et cartes de A sont perdues 3. Vérifier cartes[A] vide
Composants	Joueur, Plateau
Diagramme réf.	Figure 7 - Gestion banqueroute / faillite
Résultat attendu	cartes[A] = vide après la faillite

TI-B13 - Carte déplacement : joueur téléporté et action de la case cible déclenchée

Entrées	Joueur A à la case 10 — Carte Déplacement vers case 25
Étapes	1. Carte.action() → bouger_joueur(A, case 25) appelé 2. case[25].action() exécuté 3. Vérifier position(A) = 25 et effet de la case 25 appliqué
Composants	Carte, Joueur, Plateau, Case
Diagramme réf.	Figure 8 - Tirage d'une carte aléatoire
Résultat attendu	position(A) = 25 ; action() de la case 25 déclenchée

TI-B14 - Carte déplacement vers case départ : somme fixe créditée

Entrées	Joueur A — Carte Déplacement vers case départ
Étapes	1. Carte.action() → bouger_joueur(A, caseDepart) 2. CaseDepart.action() → donner_capital(A, montant) 3. Vérifier capital(A)
Composants	Carte, CaseDepart, Joueur, Plateau
Diagramme réf.	Figure 8 - Tirage d'une carte aléatoire
Résultat attendu	capital(A) = ancien + CaseDepart.montant ; position(A) = case 0

TI-B15 - Carte double gain : banque paie la mise si victoire au mini-jeu

Entrées	Joueur A : carte_double_gain dans cartes[A] — A joue mini-jeu (mise=100) et gagne
Étapes	1. Avant gamble() : A active carte_double_gain 2. MiniJeu.jouer() → A gagne 3. repartition() : A reçoit mise(B) = 100 € (joueur) + 100 € (banque) 4. Carte consommée
Composants	Carte, MiniJeu, Joueur, Plateau
Diagramme réf.	Figure 8 - Tirage d'une carte aléatoire
Résultat attendu	capital(A) += 200 € total (mise x 2) ; carte absente de cartes[A]

TI-B16 - Arrivée sur case prison : emprisonnement direct sans case départ

Entrées	Joueur A hors prison — Jet amène A sur la case prison
Étapes	1. bouger_joueur(A, casePrison) 2. CasePrison.action() → position(A) = case_prison 3. liste_tour[A] = 0 ; compteur_double = 0 4. Vérifier : donner_capital() NON appelé (case départ non franchie)
Composants	CasePrison, Joueur, Plateau
Diagramme réf.	Figure 9 - Case prison
Résultat attendu	A emprisonné ; liste_tour[A] = 0 ; capital inchangé (pas de bonus départ)

TI-B17 - Tour en prison sans sortie : liste_tour incrémenté

Entrées	A en prison — liste_tour[A] = 0 — Jet = non-double — Aucune carte sortie prison
Étapes	1. tour() → lancer_de() → pas de double 2. CasePrison.action() : pas de double, pas de carte 3. liste_tour[A] += 1 4. TurnManager passe au joueur suivant
Composants	CasePrison, Joueur, Plateau
Diagramme réf.	Figure 9 - Case prison
Résultat attendu	liste_tour[A] = 1 ; position(A) inchangée ; tour passé au suivant

TI-B18 - Sortie de prison par double aux dés

Entrées	A en prison — liste_tour[A] = 1 — Jet = double (3+3)
Étapes	1. tour() → lancer_de() = 3+3 → double détecté 2. CasePrison.action() → sortie de prison 3. bouger_joueur(A, case_actuelle + 6) 4. liste_tour[A] = 0
Composants	CasePrison, Joueur, Plateau
Diagramme réf.	Figure 9 - Case prison
Résultat attendu	A libéré ; position(A) = case prison + 6 ; liste_tour[A] = 0

TI-B19 - Sortie de prison après 3 tours complets

Entrées	A en prison — liste_tour[A] = 2 — Tour 3 : pas de double, pas de carte
Étapes	1. tour() → pas de double, pas de carte 2. CasePrison.action() → liste_tour[A] atteint 3 3. Sortie automatique → bouger_joueur(A, case_cible) 4. liste_tour[A] = 0
Composants	CasePrison, Joueur, Plateau
Diagramme réf.	Figure 9 - Case prison
Résultat attendu	A libéré à la fin du 3e tour en prison ; liste_tour[A] = 0

TI-B20 - Sortie de prison via carte sortie de prison

Entrées	A en prison — cartes[A] contient carte_sortie_prison — Début de tour
Étapes	1. CasePrison.action() → carte_sortie_prison détectée 2. A confirme l'utilisation 3. carte retirée de cartes[A] 4. bouger_joueur(A, case_cible)
Composants	CasePrison, Carte, Joueur
Diagramme réf.	Figure 9 - Case prison
Résultat attendu	A libéré immédiatement ; carte absente de cartes[A] ; déplacement normal

TI-B21 - Joueur en prison : ne se déplace pas durant la détention

Entrées	A en prison — Deux tours sans sortie (pas de double, pas de carte)
Étapes	1. Tour 1 : liste_tour[A] = 1 → joueur suivant 2. Tour 2 : liste_tour[A] = 2 → joueur suivant 3. Vérifier position(A) inchangée
Composants	CasePrison, Joueur, Plateau
Diagramme réf.	Figure 9 - Case prison
Résultat attendu	position(A) = case_prison (inchangée) ; liste_tour[A] = 2

TI-B22 - Case événement avec 4 joueurs : mini-jeu à 4

Entrées	4 joueurs actifs — Joueur actif atterrit sur CaseEvenement
Étapes	1. bouger_joueur(actif, caseEvenement) 2. CaseEvenement.action() → nb joueurs = 4 3. choix_minijeu() → sélectionne MiniJeu compatible 4 joueurs 4. notifier tous les joueurs → MiniJeu.jouer(j1, j2, j3, j4)
Composants	CaseEvenement, MiniJeu, Joueur
Diagramme réf.	Figure 6 - Case événement (tous les joueurs)
Résultat attendu	MiniJeu.jouer(j1, j2, j3, j4) appelé avec 4 participants

TI-B23 - Case événement avec 3 joueurs : mini-jeu à 3

Entrées	3 joueurs actifs — Joueur actif atterrit sur CaseEvenement
Étapes	1. CaseEvenement.action() → nb joueurs = 3 2. choix_minijeu() → MiniJeu à 3 joueurs 3. MiniJeu.jouer(j1, j2, j3)
Composants	CaseEvenement, MiniJeu, Joueur
Diagramme réf.	Figure 6 - Case événement (tous les joueurs)
Résultat attendu	MiniJeu.jouer(j1, j2, j3) appelé avec 3 participants

4.5 Gestion des mini-jeux (12 tests)

TI-M01 - Lancement d'un mini-jeu 1v1 depuis le plateau

Entrées	CasePropriete : propriétaire = B — Joueur A (JV) atterrit dessus — mise choisie par B
Étapes	1. CasePropriete.action() → propriétaire = B ≠ A 2. B choisit mise dans [min, max] 3. gamble() → MiniJeu instancié 4. MiniJeu.jouer(A, B) appelé
Composants	CasePropriete, MiniJeu, Joueur
Diagramme réf.	Figure 5 - Déclenchement d'un mini-jeu (1v1)
Résultat attendu	MiniJeu instancié et lancé sans erreur ; les deux joueurs entrent en phase de jeu

TI-M02 - Affichage du récapitulatif des règles avant le mini-jeu

Entrées	Mini-jeu sur le point d'être lancé
Étapes	1. Avant MiniJeu.jouer(), afficher écran des règles 2. Afficher but du jeu + touches clavier 3. Les joueurs confirment (touche Entrée ou bouton) 4. Jeu démarre
Composants	MiniJeu
Diagramme réf.	Figure 5 - Déclenchement d'un mini-jeu (1v1)
Résultat attendu	Écran de règles affiché avant le lancement ; jeu démarre après confirmation

TI-M03 - Déroulement complet d'un mini-jeu simultané (Pong)

Entrées	Mini-jeu Pong — 2 joueurs — Jeu jusqu'à la conclusion
Étapes	1. MiniJeu.jouer(A, B) → boucle de jeu temps réel 2. Un joueur atteint le score gagnant 3. Résultat retourné : Joueur[] gagnant = [A]
Composants	MiniJeu (Pong), Joueur
Diagramme réf.	Figure 5 - Déclenchement d'un mini-jeu (1v1)
Résultat attendu	Gagnant clairement désigné ; résultat retourné au moteur principal via repartition()

TI-M04 - Déroulement complet d'un mini-jeu tour par tour (Puissance 4)

Entrées	Mini-jeu Puissance 4 — 2 joueurs
Étapes	1. MiniJeu.jouer(A, B) → alternance des coups 2. J1 joue → J2 joue 3. Alignement ou grille pleine détecté 4. Résultat retourné
Composants	MiniJeu (Puissance4), Joueur
Diagramme réf.	Figure 5 - Déclenchement d'un mini-jeu (1v1)
Résultat attendu	Gagnant désigné (ou égalité si grille pleine) ; résultat retourné au moteur

TI-M05 - Résultat du mini-jeu transmis correctement au plateau

Entrées	Mini-jeu Pong terminé — A gagne — mise = 100 €
Étapes	1. MiniJeu.jouer() → retourne [gagnant = A] 2. CasePropriete.repartition() appelé avec résultat 3. enlever_capital(B, 100) ; donner_capital(A, 100) 4. update() → retour plateau
Composants	MiniJeu, CasePropriete, Plateau
Diagramme réf.	Figure 5 - Déclenchement d'un mini-jeu (1v1)
Résultat attendu	Plateau reçoit le bon gagnant ; capital(A) += 100 ; capital(B) -= 100

TI-M06 - Retour au plateau : état du plateau inchangé après mini-jeu

Entrées	Partie à 3 joueurs — B possède case 7 (Pong) — A tombe dessus et gagne
Étapes	1. Sauvegarder positions[] et propriétaires[] avant le mini-jeu 2. Mini-jeu joué jusqu'au bout 3. update() → retour plateau 4. Vérifier positions[] et propriétaires[] identiques
Composants	MiniJeu, Joueur, Plateau
Diagramme réf.	Figure 5 - Déclenchement d'un mini-jeu (1v1)
Résultat attendu	positions[] et propriétaires[] inchangés ; seuls les capitaux ont changé

TI-M07 - Soldes mis à jour immédiatement au retour plateau

Entrées	Mini-jeu terminé — Transferts effectués
Étapes	1. repartition() applique les transferts 2. update() restitue la main au Plateau 3. Plateau re-render : vérifier affichage des capitaux
Composants	Joueur, Plateau
Diagramme réf.	Figure 5 - Déclenchement d'un mini-jeu (1v1)
Résultat attendu	Capitaux affichés = valeurs après transferts (0 délai visible)

TI-M08 - Lancement successif de mini-jeux sans fuite mémoire

Entrées	5 mini-jeux lancés successivement (types différents) — Surveillance Valgrind
Étapes	1. Mini-jeu 1 instancié → joué → détruit → retour plateau 2. Répéter 5 fois avec des types différents 3. Valgrind : vérifier 0 fuite mémoire entre chaque transition
Composants	MiniJeu, Plateau
Diagramme réf.	
Résultat attendu	Valgrind : 0 fuite mémoire, 0 erreur de désallocation ; jeu stable

TI-M09 - Mini-jeu événement : redistribution %/% correcte

Entrées	4 joueurs — CaseEvenement — pot = 200 € — Classement : J1, J2, J3, J4
Étapes	1. MiniJeu.jouer(j1, j2, j3, j4) → classement [J1, J2, J3, J4] 2. distribution() → donner_capital(J1, 150) ; donner_capital(J2, 50) 3. J3, J4 : aucun transfert
Composants	CaseEvenement, MiniJeu, Joueur, Plateau
Diagramme réf.	Figure 6 - Case événement (tous les joueurs)
Résultat attendu	capital(J1) += 150 (3/4) ; capital(J2) += 50 (1/4) ; total redistribué = 200 €

TI-M10 - Mini-jeu sans composante aléatoire : résultat reproductible

Entrées	Mini-jeu Pong — Séquence d'entrées identique S rejouée 2 fois
Étapes	1. Partie 1 : jouer avec séquence S → noter gagnant G1 2. Partie 2 : rejouer avec séquence S → noter gagnant G2 3. Comparer G1 et G2
Composants	MiniJeu (Pong)
Diagramme réf.	
Résultat attendu	G1=G2 : même gagnant pour la même séquence (pas d'aléatoire)

TI-M11 - Gestion des entrées simultanées de deux joueurs

Entrées	Mini-jeu simultané Pong — J1 appuie sur Haut, J2 appuie simultanément sur Bas
Étapes	1. Boucle de jeu : traitement des événements SFML 2. J1 → sf::Keyboard::Up détecté 3. J2 → sf::Keyboard::Down détecté simultanément 4. Les deux actions traitées dans la même frame
Composants	MiniJeu, Plateau
Diagramme réf.	Figure 5 - Déclenchement d'un mini-jeu (1v1)
Résultat attendu	Les deux inputs traités sans conflit ; aucun crash ; jeu continue normalement

TI-M12 - Ajout d'un nouveau mini-jeu via héritage : moteur inchangé

Entrées	Nouvelle classe NouveauJeu : public MiniJeu — Assignée à une CasePropriete
Étapes	1. NouveauJeu implémente toutes les méthodes virtuelles de MiniJeu 2. CasePropriete.jeu = new NouveauJeu() 3. gamble() → MiniJeu* ptr jouer() appelé polymorphiquement 4. Vérifier : aucune modification dans Plateau.cpp, CasePropriete.cpp
Composants	MiniJeu (abstrait), NouveauJeu, CasePropriete
Diagramme réf.	
Résultat attendu	NouveauJeu lancé normalement via MiniJeu* ; 0 modification dans le moteur existant

4.6 Robustesse (9 tests)

TI-R01 - Lancement d'un mini-jeu en moins de 3 actions

Entrées	Partie en cours — A tombe sur case possédée par B
Étapes	Action 0 : A atterrit (automatique) Action 1 : B confirme la mise Action 2 : A confirme le duel → Mini-jeu lancé
Composants	CasePropriete, MiniJeu
Diagramme réf.	Figure 5 - Déclenchement d'un mini-jeu (1v1)
Résultat attendu	Mini-jeu démarre en ≤ 3 actions (clics/frappes) depuis l'atterrissage

TI-R02 - Temps de chargement transition plateau mini-jeu ; 2 secondes

Entrées	Partie en cours — Assets chargés par ResourceManager — Mini-jeu déclenché
Étapes	1. t0 = chrono : :now() au déclenchement 2. Transition plateau → mini-jeu 3. t1 = chrono : :now() à l’affichage de l’écran de règles → dt1 = t1-t0 4. Mini-jeu terminé → t2 = chrono : :now() à l’affichage du plateau → dt2 = t2-fin_mini-jeu
Composants	MiniJeu, Plateau
Diagramme réf.	Figure 5 - Déclenchement d’un mini-jeu (1v1)
Résultat attendu	dt1 ; 2s ET dt2 ; 2s ; mesure répétée sur 10 transitions consécutives

TI-R03 - Session de 60 minutes sans erreur fatale ni fuite mémoire

Entrées	Partie à 4 joueurs — Session continue de 60 min — Valgrind actif
Étapes	1. Démarrer la session 2. Jouer en continu 60 min (mini-jeux, cartes, prison, faillites) 3. Surveiller Valgrind : fuites mémoire, invalid reads/writes 4. Vérifier logs d’erreur SFML
Composants	Tous les composants
Diagramme réf.	
Résultat attendu	0 erreur fatale ; 0 fuite mémoire Valgrind ; 0 crash SFML sur 60 min

TI-R04 - Asset manquant au chargement : message d’erreur propre, pas de crash

Entrées	Fichier assets/textures/plateau.png supprimé avant le lancement
Étapes	1. Lancer le jeu 2. ResourceManager : :load() → fichier introuvable 3. Vérifier : message d’erreur affiché (stderr ou UI) 4. Vérifier : programme ne crashe pas (pas de SIGSEGV)
Composants	Plateau
Diagramme réf.	
Résultat attendu	Message d’erreur explicite affiché ; programme stable ou dégradé gracieusement

TI-R05 - Saisie invalide pendant un mini-jeu : comportement stable

Entrées	Mini-jeu Pong en cours — Saisies : touches non mappées + combinaisons multi-touches
Étapes	1. Appuyer sur F12, Suppr, Ctrl+Alt+Del pendant le jeu 2. Appuyer sur 10 touches simultanées 3. Observer état du mini-jeu après chaque saisie invalide
Composants	MiniJeu
Diagramme réf.	
Résultat attendu	Mini-jeu continue normalement ; aucun crash ; état cohérent

TI-R06 - Interruption prématurée : vainqueur désigné par patrimoine

Entrées	Partie à 4 joueurs actifs — Arrêt manuel déclenché
Étapes	1. Arrêt manuel déclenché mid-game 2. fin() calcule patrimoine = capital + val_proprietes pour chaque joueur 3. Tri décroissant → annonce du classement
Composants	Plateau, Joueur
Diagramme réf.	Figure 3 - Déroulement d'un tour complet
Résultat attendu	Classement annoncé sans erreur ; joueur avec le plus grand patrimoine déclaré vainqueur

TI-R07 - Partie complète à 2 joueurs de bout en bout

Entrées	2 joueurs — Partie jusqu'à faillite d'un joueur
Étapes	1. Initialisation → tours joués jusqu'à faillite d'un joueur 2. fin() → vainqueur annoncé 3. Vérifier : aucune erreur fatale, aucune fuite mémoire
Composants	Tous les composants
Diagramme réf.	
Résultat attendu	Partie complète sans erreur fatale ; vainqueur correctement désigné

TI-R08 - Partie complète à 3 joueurs de bout en bout

Entrées	3 joueurs — Partie jusqu'à 2 faillites
Étapes	1. Initialisation → tours joués jusqu'à 2 faillites 2. fin() → vainqueur annoncé 3. Vérifier stabilité
Composants	Tous les composants
Diagramme réf.	
Résultat attendu	Partie complète sans régression ; vainqueur correctement désigné

TI-R09 - Partie complète à 4 joueurs de bout en bout

Entrées	4 joueurs — Partie jusqu'à 3 faillites
Étapes	1. Initialisation → tours joués jusqu'à 3 faillites 2. fin() → vainqueur annoncé 3. Vérifier stabilité
Composants	Tous les composants
Diagramme réf.	
Résultat attendu	Partie complète sans régression ; vainqueur correctement désigné

Conclusion

Nous avons abordé dans ce rapport la modélisation globale de notre jeu GambleStyle via le langage UML. En conclusion de ce cahier de spécifications, nous voulons mettre un point d'honneur sur la modélisation de notre jeu. Ce cahier de spécifications résume en quelque sorte notre avancement sur le projet à l'instant du rendu et est le fruit de notre travail. Bien que nous soyons conscient que la modélisation proposée ne soit pas l'unique, nous pensons que cette modélisation nous permettra d'avancer dans la suite du projet plus sereinement.